UNIVERSITY OF RWANDA.
COLLEGE OF SCIENCE AND Technology
Computer and Software Engineering.

on 17th March 2026
Reg No: 223008047

MOBILE APPLICATION SYSTEMS AND DESIGN.

(1)

POST MANAGER – FLUTTER API INTEGRATION LAB:

1. DEPENDENCIES USED AND WHY;

Dependencies	PURPOSE.
http	making API requests to JSONplaceholder (GET, POST, PUT, DELETE).
Connectivity_Plus	checking internet connectivity before making API calls.
provider	state management across screens.
google_fonts	Professional typography.
shimer	Beautiful loading animations.
flutter_staggered_animations	Smooth list animations.

POST MANAGER - FLUTTER API INTEGRATION LAB

2 API EXCEPTION HANDLING

EXCEPTION TYPE	HANDLING TYPE METHOD
No internet	- check with connectivity_plus - show user friendly error massage.
Network Timeout	- set timeout (10 seconds) - catch TimeoutException. - suggest checking connection.
HTTP Errors (4xx, 5xx)	- check status codes. - parse error messages. - display appropriate feedback.
JSON parsing Errors.	- Try-catch during decoding. - Validate data structure. - show fallback UI.

IMPLEMENTATION EXAMPLE :

```dart
try ~~try~~ {
  if (! await hasInternet()) throw 'No connection';
  final response = await http.get(Url).timeout(10.seconds);
  if (response.statusCode == 200) {
    return Post.fromJSON(JSON.decode(response.body));
  } Else {
    throw 'server Error: ${response.statusCode}';
  }
```

② (following):

```
} on TimeoutException catch (_) {
  throw 'request timed out';
} catch (e) {
  throw 'Error: $e';
}
```

③ FUTURE WIDGET EXPLANATION

Part III: Which Future ~~Builder~~ widget did you use?

⇒ the FutureBuilder widget.

In my Post Manager application, I used FutureBuilder the flutter widget that builds itself based on the latest snapshot of interaction with a Future.

dart code example.

```
FutureBuilder<List<Post>>(
  future: _apiservice.fetchPosts(), // the Future that fetches data.
  builder: (BuildContext Context, AsyncSnapshot<List<Post>> snapshot) {
    // UI Builder function that rebuilds when future state changes.
    // Return different UI based on snapshot state.
  },
)
```

Why FutureBuilder?

Because of:

- It automatically handles different states (loading, error, data).
- No need for manual setState() calls.
- clean and simple code.

Part II

UI parts that depends on the FutureBuilder:

Future state	What happens	What user sees
Waiting	API call in progress	Loading spinner or shimmer effe
Error	API failed	Error message + Retry button
Empty	No posts returned	No post message + Create butto
Data	post received	List of posts with titles

~~Part III dart simple code example:~~

PART III dart code example.

```
FutureBuilder < List <Post>>(
  future: - apiservice.fetchposts(),
  Builder: (Context, snapshot) {
   // Loading UI
   if (snapshot.ConnectionState == ConnectionState.waiting) {
        return center (child: circularprogressIndicator());
      }
   // Error UI.
   if (snapshot.hasError) {
      return center( child: Text ('Error: ${snapshot.error}' ));
        }
  // Empty UI
   if (!snapshot.hasdata || snapshot.data!.isempty) {
        return Center (child: Text ('No post found'));
        }
  // Data UI.
  return ListView.builder(
      itemCount: snapshot.data!.length, itembuilder: (Context, index) {
};  ); }; return ListTile ( title: Text (snapshot.data![index].title), );
```